

# A Built-In Self-Repair With Maximum Fault Collection and Fast Analysis Method for HBM

Joonsik Yoon<sup>ib</sup>, Hayoung Lee<sup>ib</sup>, Youngki Moon<sup>ib</sup>, Seung Ho Shin<sup>ib</sup>, and Sungho Kang<sup>ib</sup>, *Senior Member, IEEE*

**Abstract**—High bandwidth memory (HBM) represents a significant advancement in memory technology, requiring quick and accurate data processing. Built-in self-repair (BISR) is crucial for ensuring high-capacity and reliable memories, as it automatically detects and repairs faults within memory systems, preventing data loss and enhancing overall memory reliability. The proposed BISR aims to **enhance the repair rate and reliability by using a content-addressable memory structure that operates effectively in both offline and online modes**. Furthermore, **a new redundancy analysis algorithm reduces both analysis time and area overhead by converting fault information into a matrix format and focusing on fault-free areas for each repair solution**. Experimental results demonstrate that the proposed BISR improves repair rates and **derives a final repair solution immediately after the test sequences are completed**. Moreover, hardware comparisons have shown that the proposed approach reduces the area overhead as memory size increases. Consequently, the proposed BISR enhances the overall performance of BISR and the reliability of HBM.

**Index Terms**—Analysis time, built-in redundancy analysis (BIRA), built-in self-repair (BISR), content-addressable memory (CAM), repair rate.

## I. INTRODUCTION

MEMORY is expected to expand continuously due to the increasing demand for data processing in big data applications and various other fields. High bandwidth memory (HBM) represents a major advancement in memory technology, utilizing a 3D-stacked architecture and through-silicon via (TSV) technology [1], [2]. These structures significantly expand memory capacity and bandwidth, addressing the demands of high-performance computing applications. HBM plays a crucial role in enhancing data throughput and performance in high-performance and graphics computing applications [3], [4]. Fig. 1(a) shows the structure of HBM, with DRAM dies stacked, a logic die at the bottom, and

Received 12 June 2024; revised 4 October 2024; accepted 12 November 2024. Date of publication 14 November 2024; date of current version 23 April 2025. This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea Government (MSIT) under Grant 2022R1A2B5B03002504. This article was recommended by Associate Editor G. D. Natale. (*Corresponding author: Sungho Kang.*)

Joonsik Yoon, Youngki Moon, and Seung Ho Shin are with the Computer Systems Reliable SoC Laboratory, Department of Electrical and Electronic Engineering, Yonsei University, Seoul 03722, South Korea (e-mail: junsikyoon@yonsei.ac.kr; yk.moon@yonsei.ac.kr; shin080314@yonsei.ac.kr).

Hayoung Lee is with the Department of Intelligence Semiconductor Engineering, Ajou University, Suwon 16499, Gyeonggi, South Korea (e-mail: hylee@ajou.ac.kr).

Sungho Kang is with the Department of Electrical and Electronic Engineering, Yonsei University, Seoul 03722, South Korea (e-mail: shkang@yonsei.ac.kr).

Digital Object Identifier 10.1109/TCAD.2024.3499903

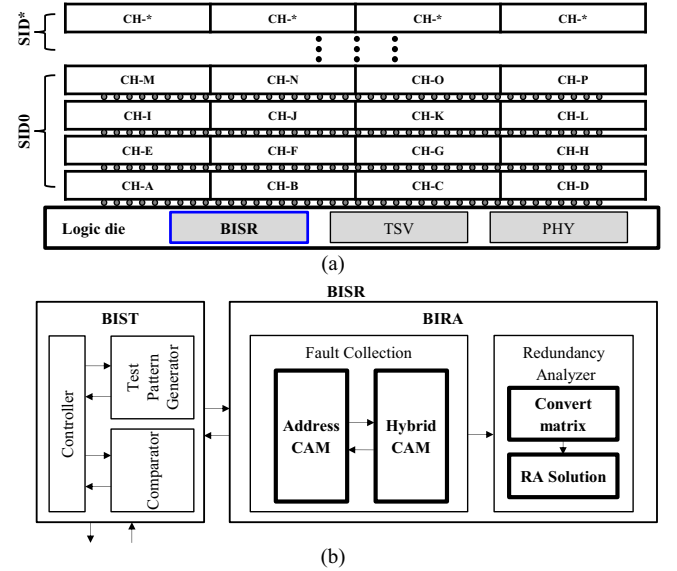


Fig. 1. HBM. (a) Structure. (b) Block diagram of proposed BISR.

interconnections made through TSV. In HBM3, a single stack ID consists of four DRAM dies, which together form 16 independent channels [5], [6], [7]. The logic die is responsible for executing data processing and management functions within the DRAM dies and includes built-in self-repair (BISR) circuits to improve reliability [8].

As the capacity of DRAM expands and scaling advances, the probability of fault occurrence in the DRAM increases due to rising interference between cells and circuit complexity, leading to a degradation in yield and reliability [9]. To address these issues, there are two main methods for repairing memory systems [10]. The first method involves using automatic test equipment (ATE) to detect faulty cells in memory and storing the information within the internal fault bitmap memory of the ATE. Subsequently, the optimal repair solution is derived through redundancy analysis (RA) to repair the memory cells. This method is primarily used at the wafer level to repair initial manufacturing defects [11], [12], [13], [14], [15], [16]. The second method utilizes BISR circuits at both the package level and the system-in-package (SiP) level, enabling faults to be detected and repaired without relying on ATE. As shown in Fig. 1(b), the built-in self-test (BIST) generates test patterns to detect faults within the DRAM, whereas the built-in RA (BIRA) focuses on the repair analysis based on the spare-based

architecture of the DRAM. Thus, the role of BISR is to enhance the yield and the reliability of DRAM by detecting and repairing faults.

An ideal BISR is characterized by an optimal repair rate, no analysis time, and no area overhead. The repair rate for memories is defined as the proportion of successfully repaired memories out of those tested. The analysis time is the duration required to derive a repair solution. The area overhead means designing with minimal area usage within the limited chip area available. Thus, these key features are closely related to manufacturing costs.

Various research focuses on improving these key features to align better with the ideal BISR characteristics. CRESTA [17] achieves an optimal repair rate by using a subanalyzer to find all repair solutions, and repairs are analyzed quickly even during memory testing, thus offering the advantage of fast analysis time. However, as the number of spares increases, the required number of subanalyzers also dramatically increases, leading to a significant increase in area overhead. ESP [18] adopts a dynamic BIRA approach to conduct memory repairs. Repair registers are used to reduce hardware overhead. However, ESP cannot achieve an optimal repair rate. ISF [19] algorithm progresses with must-repair analysis during fault collection. The repair solution search process stops after the first repair solution is found. ISF is capable of achieving an optimal repair rate and has a smaller area overhead compared to CRESTA. However, the analysis time of ISF is significantly slow due to numerous backtracking decisions.

Many studies have explored the use of content-addressable memory (CAM), known for its fast address comparison capability, to enhance the overall performance of BISR. Consequently, CAM-based BISR has been widely adopted as the conventional BISR in various research [20], [21], [22], [23], [24]. SFCC [20] achieves an optimal repair rate and lowers area overhead compared to the previous BISRs. A line-based search for RA is employed by the algorithm, which results in faster analysis time compared to the ISF method. However, as the number of lines with multiple faults increases, analysis time can increase exponentially. BRANCH [21] achieves an optimal repair rate and offers faster analysis time compared to SFCC while maintaining similar area overhead. Cho et al. [22] proposed reducing area overhead in CAM by storing cross-fault information in the spare line of DRAM. However, the disadvantage is the need for additional data management and control in the spare line of DRAM. Han et al. [23] proposed improving the repair rate by adding spare cells composed of SRAM to the logic die. To maintain a similar area overhead to [21], the structure of CAM is simplified by storing only pivot fault information. Therefore, when fault information is complex, there is a disadvantage in that the repair rate decreases. Lee et al. [24] proposed a memory repair process that quickly reconfigures fault data when new fault information occurs, enabling an optimal repair solution during test sequences. However, when all fault information is stored in the CAM, additional buffer space may be required for reconfiguration, resulting in increased area overhead for the repair analyzer.

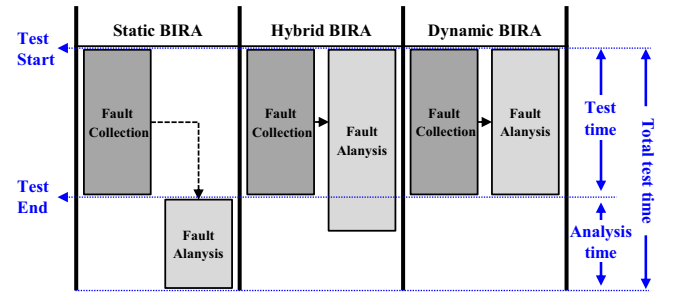


Fig. 2. BIRA operation classification.

Several different approaches have been conducted in addition to previously researched methods. In [25], BISR utilizing configurable redundancies based on fault patterns has been proposed, where the redundant SRAM module is applied exclusively to the logic die. In [26], BISR has been proposed in which fault information is stored in a fault-free region of DRAM to reduce the area overhead.

The proposed BISR is specifically designed for HBM, enhancing the yield and the reliability through the utilization of CAM as redundancy resources. As HBM progresses through wafer-level, package-level, and SiP processes, the possibility of additional faults occurring increases due to the complexity of the manufacturing processes and the high-capacity memory. In particular, since HBM is a high-cost product, improving yield is critical. The CAM can be reused as redundancy resources that are available for all channels, enabling efficient repair when faults occur in a specific channel. Moreover, increasing the CAM size by utilizing the available area in the logic die can lead to further enhancements. Consequently, the proposed BISR can significantly contribute to the reduction of manufacturing costs by improving yield.

A new BISR is proposed to enhance the repair rate and the reliability by using a CAM structure that effectively operates in both offline and online modes. The overall performance of BISR is enhanced by a new RA algorithm, where more faults are collected through CAM reuse, and fault information is converted into a matrix format, allowing the repair analyzer to focus exclusively on fault-free areas. This approach provides a fast repair solution, significantly reducing analysis time, complexity, and area overhead. Therefore, the repair process is made more efficient, and the reliability and cost-effectiveness of HBM are improved.

## II. BACKGROUND

### A. Operation Classification

As shown in Fig. 2, the process of deriving the repair solution for memory can be categorized into static BIRA, hybrid BIRA, and dynamic BIRA. In static BIRA, fault information is stored in the fault collection while the test is being conducted. Upon completion of the test, this information is processed by a repair analyzer to derive the repair solution. In static BIRA, the repair analysis is performed utilizing all stored fault information, which is advantageous for achieving an optimal repair rate. However, a disadvantage is the additional time required after testing. In hybrid BIRA, the analysis time is

improved by analyzing fault information in advance when must-repair conditions are satisfied. In dynamic BIRA, fault information collected during testing is immediately analyzed to derive the repair solution, but achieving an optimal repair rate is highly challenging due to the continuous addition of fault information. However, dynamic BIRA has the advantage of not requiring additional analysis time compared to static BIRA, which can lead to reduced total testing costs. The proposed idea is classified as dynamic BIRA, which can reduce the test time while achieving an optimal repair rate.

### B. Fault Classification Using CAM

The CAM is widely used in BISR due to its ability to perform a lookup-table function within a single clock cycle using the dedicated parallel comparison circuitry [27]. This high-speed capability makes it ideal for fault collection in BISR, enabling efficient identification and management of fault addresses. The repair process is significantly enhanced by the CAM's functionality in ensuring fast fault detection, making it particularly effective for real-time fault management in BISR. On the other hand, in [28], the CAM is utilized for the purpose of an address remapping scheme in online mode.

The BISR, which utilizes the CAM for fault collection, classifies faults as pivot or nonpivot. If a newly detected fault has a unique row and column address compared to previously collected faults, it is classified as a pivot fault. Therefore, pivot faults do not share row and column addresses with each other. All faults except pivot faults are defined as nonpivot faults. After pivot faults are decided, every nonpivot fault should share its row or column address with at least one of the pivot faults.

The **must-repair algorithm** is implemented based on the availability of spare rows and columns in the DRAM. If a row contains more faults than the number of available spare columns, it is replaced by a spare row. Similarly, if a column has more faults than the number of available spare rows, it is replaced by a spare column.

The **early termination** algorithm can be executed during testing when it is determined that repairs are impossible due to a lack of repair resources, thus preventing unnecessary analysis time. Therefore, total test time can be significantly reduced if it is determined whether the memory is repairable before the repair analysis is initiated.

The evaluation through a simulation of memory redundancy algorithms in [29] demonstrates that high hardware costs are incurred by collecting faults necessary to achieve an optimal repair rate in BISR. However, the proposed BISR introduces optimized fault classification and collection methods, which can reduce hardware costs, resulting in a more efficient repair process and achieving an optimal repair rate.

## III. PROPOSED IDEA

### A. Overview

In this article, a new BISR called RECAM (BISR with maximum fault collection and fast analysis method) is proposed to enhance the performance of BISR by maximizing fault collection and streamlining repair analysis through the reuse

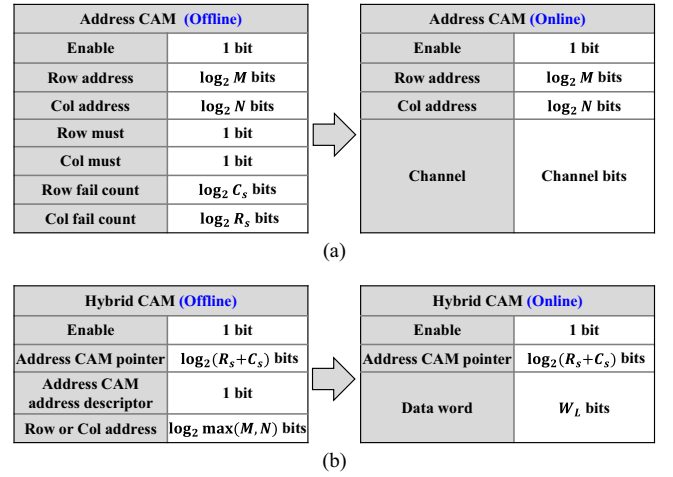


Fig. 3. Structure of CAM in offline and online modes. (a) Address CAM. (b) Hybrid CAM.

of CAM and efficient utilization in both offline and online modes.

RECAM develops new implementations in three key features to maximize the performance of BISR. **First, the reuse of CAM**, typically used for fault collection in offline mode, facilitates the additional storage of **pivot fault information**, significantly enhancing the repair rate compared to CAM-based BISRs. **Second**, converting fault information into a **matrix format** allows for the representation of all fault data. A fast analysis time is enabled by analyzing only the fault-free areas based on this information. **Third, reusing CAM as redundancy resources in online mode** not only enhances memory yield but also ensures the reliability of the memory. Consequently, utilizing RECAM contributes to a substantial enhancement of the overall performance of the BISR through the new repair algorithm.

### B. CAM Structure

To efficiently operate in both offline and online modes, the proposed structure of CAM is divided into the **Address CAM** and the **Hybrid CAM**, each configured to perform different roles within the same structure. As a result, the burden of area overhead is minimized by reusing the CAMs. The structure of CAM is shown in Fig. 3, representing the number of spare rows ( $R_s$ ) and the number of spare columns ( $C_s$ ) for memory size  $M \times N$ . In both offline and online modes, pivot faults are stored in the Address CAM. Particularly in online mode, the Address CAM is utilized to store the fault addresses for redundancy resources, performing functions, such as an enable bit, row address ( $\log_2 M$ ) bits, column address ( $\log_2 N$ ) bits, and channel bits, similar to the functions used in offline mode. The Hybrid CAM is used for fault collection of nonpivot addresses in offline mode, whereas in online mode, it is utilized for storing the data word and transferring the data word to the output multiplexer. The pointer function is served by the address CAM pointer bits in both offline and online modes. The address descriptor bit and address ( $\log_2 \max(M, N)$ ) bits are altered as the data word ( $W_L$ ) bits for online mode. For example, in **HBM3**, the

TABLE I  
NUMBER OF ADDRESSES IN THE CAM

| $R_s$ | $C_s$ | # of addresses in Address CAM ( $R_s + C_s$ ) | # of addresses in Hybrid CAM ( $R_s(C_s - 1) + C_s(R_s - 1)$ ) |
|-------|-------|-----------------------------------------------|----------------------------------------------------------------|
| 2     | 2     | 4                                             | 4                                                              |
| 2     | 3     | 5                                             | 7                                                              |
| 3     | 3     | 6                                             | 12                                                             |
| 4     | 4     | 8                                             | 24                                                             |
| 5     | 5     | 10                                            | 40                                                             |

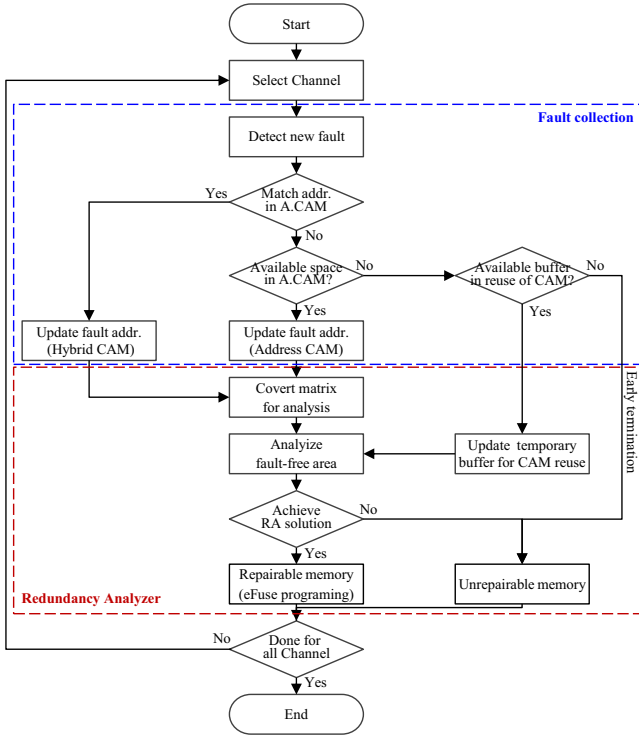


Fig. 4. Flowchart for repair analysis algorithm.

maximum address of a channel uses at least 16 bits. Therefore, in online mode, there is sufficient space to store the data word for pivot faults.

Table I provides the number of addresses that can be stored in the proposed CAM. The number of addresses in Address CAM is determined by the number of spare rows and spare columns. The relationship between the number of addresses in the Address CAM and the Hybrid CAM shows that as the number of spares increases, the number of addresses in the Hybrid CAM increases significantly more than in the Address CAM. Therefore, the data word in the Hybrid CAM can be further expanded by utilizing a larger number of addresses than in the Address CAM. In addition, there is no issue with assigning the data words in the Hybrid CAM to the addresses of the Address CAM.

### C. Fault Collection for CAM Reuse

The flowchart of the repair analysis algorithm in offline mode is illustrated in Fig. 4. In HBM, each channel is tested using BIST, and the results are compared with the expected

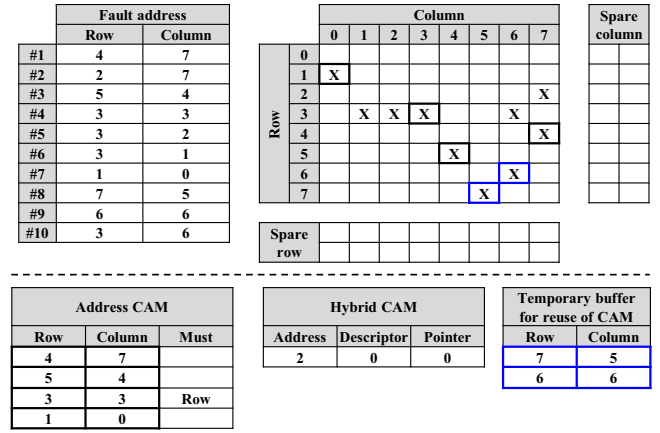


Fig. 5. Example of fault collection for CAM reuse. ( $R_s, C_s = [2, 2]$ ).

data to identify faulty addresses. When a new fault is detected, the pivot fault is updated in the Address CAM. Additionally, if newly detected faults match the fault information in the Address CAM, the faults are updated in the Hybrid CAM. If there is no additional space to store pivot faults in the Address CAM, an early termination would be triggered in CAM-based BISRs, resulting in the memory being classified as unrepairable. However, reusing CAM enhances the fault collection capacity by transferring additional pivot faults from the analyzer, thereby allowing the CAM to be utilized as redundancy resources. As a result, more faults can be repaired in comparison to CAM-based BISRs.

Fig. 5 shows an example of fault collection for CAM reuse, in a state with two spare rows and two spare columns within an  $8 \times 8$  memory. A total of ten faults are classified as either pivot or nonpivot and are accordingly updated in the Address CAM and the Hybrid CAM. The fault (4, 7) is classified as a pivot fault due to its unique row and column addresses, and its address is updated in the Address CAM. With the address for column "7" already stored in the Address CAM, the fault (2, 7) is classified as a nonpivot fault. Subsequently, the row address "2" is updated in the Hybrid CAM, with the descriptor set to "0" to indicate the row, while the column address is updated as 0 pointer bits based on the addresses stored in the Address CAM. The faults (5, 4) and (3, 3), classified as pivot faults, are updated in the Address CAM. Next, the fault (3, 2), classified as a nonpivot fault, is updated in the Hybrid CAM. On the other hand, when additional faults occur at (3, 1) while the address for row "3" has already been updated twice in the CAMs, and the number of these faults exceeds the available column spares, the must-repair condition for the row is satisfied. Therefore, there is no need to update the faults at the column addresses for row 3. Next, the fault (1, 0) is classified as a pivot fault and updated in the Address CAM. At this point, all available spaces within the Address CAM have become occupied with updated fault information. Afterward, if additional pivot faults occur, early termination would be executed in the case of CAM-based BISRs. However, reusing the CAM allows for the storage of additional pivot faults up to the capacity of  $R_s + C_s$ , which is equal to the number of



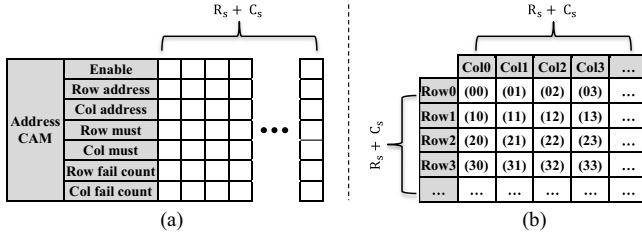


Fig. 6. Structure. (a) Address CAM. (b) Matrix conversion.

addresses in the Address CAM. Once the test ends, the faults for (7, 5) and (6, 6), which are transmitted to the temporary buffers in the repair analyzer, are programmed into the eFuses concurrently with the repair solution derived from the repair analyzer. The faults are then repaired through CAM reuse in online mode. Although the addition of temporary buffers results in increased area overhead, this increase is considered negligible since the temporary buffers are used exclusively for fault information storage without any logical operations.

#### D. Matrix Conversion for Analysis

As shown in Fig. 6(a), the Address CAM structure enables the representation of fault information obtained during memory testing in a matrix format, as illustrated in Fig. 6(b). The size of this matrix is defined as  $(R_s + C_s) \times (R_s + C_s)$  with the signal at each position composed of combinational logic gates. The rows and columns in Fig. 6(b) are based on the fault addresses of the rows and columns in the Address CAM. When the enable signal of the Address CAM is activated, specific matrix positions, such as (00), (11), (22), and (33) are activated following the storage order in the Address CAM. If a must-repair condition is satisfied, this activation extends fully to either all rows or all columns. The other signals are determined by the fault information stored in the Hybrid CAM. Each fault address is checked for a match with the addresses in the Address CAM and is converted into a pointer based on the Address CAM. Whenever new fault information is added, a matrix conversion operation becomes necessary since the fault type may change. Consequently, this conversion facilitates the verification of fault occurrences at specified matrix positions.

The matrix conversion methods can be broadly classified into five types based on the faults stored in the Address CAM and Hybrid CAM. Fig. 7 shows examples of matrix conversion methods for different fault types in a memory with two spare rows and two spare columns. The total matrix size can be represented by 16 signals. The matrix conversion method for fault information in the Address CAM is illustrated in Fig. 7(a). When the enable bit for information on three faults is enabled, the signals at (00), (11), and (22) are activated. Additionally, adding fault information at (7, 1) activates the signal at (33). Fig. 7(b) shows the matrix conversion for **must-repair conditions** based on the fault information in the Address CAM. When a must-repair condition is satisfied by column “4,” the signal for the corresponding entire column is activated, and when a must-repair condition is satisfied by row “1,” the signal for the corresponding entire row is activated.

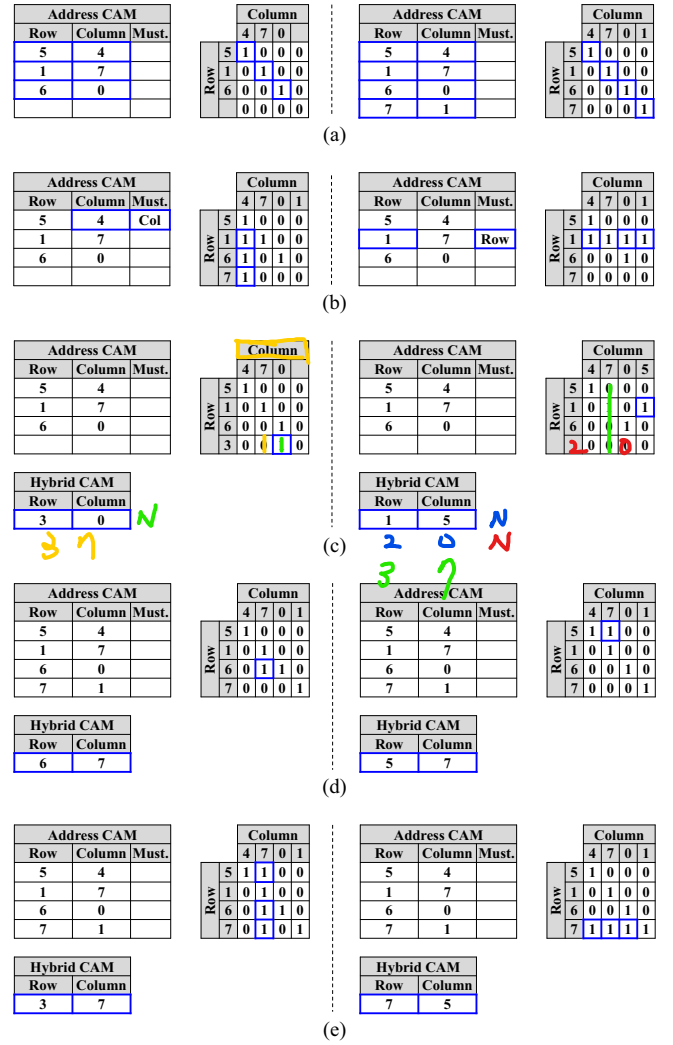


Fig. 7. Examples of matrix conversion based on fault type. (a) Enable information in Address CAM. (b) Must-repair condition. (c) Case 1 for Hybrid CAM. (d) Case 2 for Hybrid CAM. (e) Case 3 for Hybrid CAM. ( $R_s$ ,  $C_s$  = [2, 2]).

The fault information in the **Hybrid CAM** is classified into three types using pointer information based on the fault information from the Address CAM. **The first case** is when there is remaining storage space in the Address CAM. Fig. 7(c) shows that if the fault information in the Hybrid CAM is (3, 0), and since the column 0 is stored in the Address CAM, the signal for row 3 is activated in the remaining space of the matrix row, aligned with column 0. Similarly, if the fault information in the Hybrid CAM is (1, 5), the signal for column “5” is activated in the remaining space of the matrix column, aligned with row 1. **The second case** occurs when the row and column addresses of the fault information in both the Address CAM and the Hybrid CAM are completely matched. As shown in Fig. 7(d), if (6, 7) and (5, 7) are stored in the Hybrid CAM, it demonstrates that the signals in areas where both row and column addresses match the fault information in the Address CAM are activated. **The final case** occurs when **only the row addresses** of the Hybrid CAM match those of the Address CAM, or when **only the column addresses**

| Repair<br>solution lists | Spare# | #1      | #2   | #3   | #4   | #5   | #6   |
|--------------------------|--------|---------|------|------|------|------|------|
|                          |        | Row0    | Row0 | Row0 | Col0 | Col0 | Col0 |
|                          |        | Spare#1 | Row1 | Col1 | Col1 | Row1 | Row1 |
|                          |        | Spare#2 | Col2 | Row2 | Col2 | Row2 | Row2 |
|                          |        | Spare#3 | Col3 | Col3 | Row3 | Row3 | Row3 |

(a)

| #1   | Col0 | Col1 | Col2 | Col3 |
|------|------|------|------|------|
| Row0 | (00) | (01) | (02) | (03) |
| Row1 | (10) | (11) | (12) | (13) |
| Row2 | (20) | (21) | (22) | (23) |
| Row3 | (30) | (31) | (32) | (33) |

| #2   | Col0 | Col1 | Col2 | Col3 |
|------|------|------|------|------|
| Row0 | (00) | (01) | (02) | (03) |
| Row1 | (10) | (11) | (12) | (13) |
| Row2 | (20) | (21) | (22) | (23) |
| Row3 | (30) | (31) | (32) | (33) |

| #3   | Col0 | Col1 | Col2 | Col3 |
|------|------|------|------|------|
| Row0 | (00) | (01) | (02) | (03) |
| Row1 | (10) | (11) | (12) | (13) |
| Row2 | (20) | (21) | (22) | (23) |
| Row3 | (30) | (31) | (32) | (33) |

| #4   | Col0 | Col1 | Col2 | Col3 |
|------|------|------|------|------|
| Row0 | (00) | (01) | (02) | (03) |
| Row1 | (10) | (11) | (12) | (13) |
| Row2 | (20) | (21) | (22) | (23) |
| Row3 | (30) | (31) | (32) | (33) |

| #5   | Col0 | Col1 | Col2 | Col3 |
|------|------|------|------|------|
| Row0 | (00) | (01) | (02) | (03) |
| Row1 | (10) | (11) | (12) | (13) |
| Row2 | (20) | (21) | (22) | (23) |
| Row3 | (30) | (31) | (32) | (33) |

| #6   | Col0 | Col1 | Col2 | Col3 |
|------|------|------|------|------|
| Row0 | (00) | (01) | (02) | (03) |
| Row1 | (10) | (11) | (12) | (13) |
| Row2 | (20) | (21) | (22) | (23) |
| Row3 | (30) | (31) | (32) | (33) |

(b)

Fig. 8. Method for deriving the repair solution. (a) Repair solution lists. (b) Matrix conversion. ( $R_s, C_s = [2, 2]$ ).

match. In Fig. 7(e), if (3, 7) is stored in the Hybrid CAM, only column 7 is matched with the fault information in the Address CAM, resulting in the activation of all signals in that column. Similarly, if (7, 5) is stored in the Hybrid CAM, only row 7 is matched with the fault information in the Address CAM, resulting in the activation of all signals in that row. Consequently, through matrix conversion based on fault type, all fault information can be represented to derive the repair solution according to the repair resources.

### E. Fault-Free Area Analysis

The number of spare rows ( $R_s$ ) and the number of spare columns ( $C_s$ ) serve as the criteria for storing fault information in the Address CAM, and the total number of repair solutions can be defined by  $(R_s + C_s)! / \{(R_s!) \times (C_s!)\}$ . As shown in Fig. 8(a), assuming that there are two spare repair resources for each row and column, a maximum of six cases can arise in deriving the repair solution, and the possibility of repairs is judged for each case based on all fault information. Judging each case individually increases both the total test time and the overall test costs. To resolve this disadvantage, the proposed approach evaluates all cases at once to derive the repair solution.

The repair solution lists described in Fig. 8(a) are transformed into six tables by applying the matrix conversion to each repair solution, as shown in Fig. 8(b). A fast repair solution can be achieved by using NOR gates to verify signals in areas designated as fault-free according to the repair solution. For example, in the first table of Fig. 8(b), if the signals at (20), (21), (30), and (31) are verified with NOR gates and found to be fault-free, then the repair solution for rows 0 and 1 and columns 2 and 3 can be achieved by the repair analysis. In general, the repair analysis involving the verification of all nonpivot faults based on pivot faults is complex and time-consuming. However, by verifying only areas where faults should not occur, the time required to derive the repair solution can be reduced.

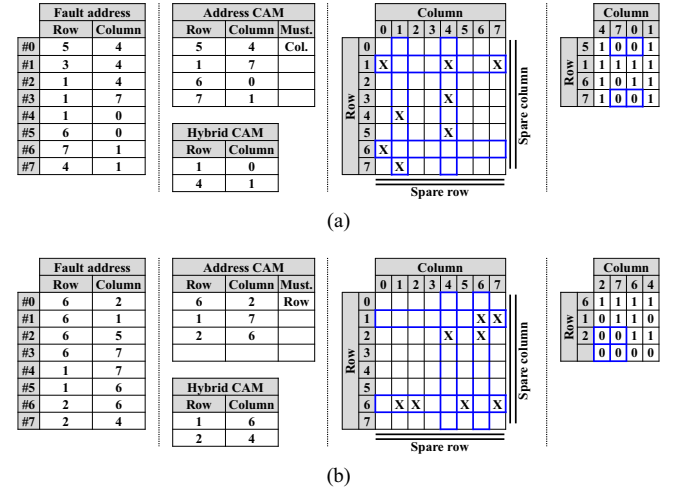


Fig. 9. Examples of repair CAM analysis through fault-free areas. ( $R_s, C_s = [2, 2]$ ). (a) Corresponding to the solution list #4. (b) Corresponding to the solution list #1.

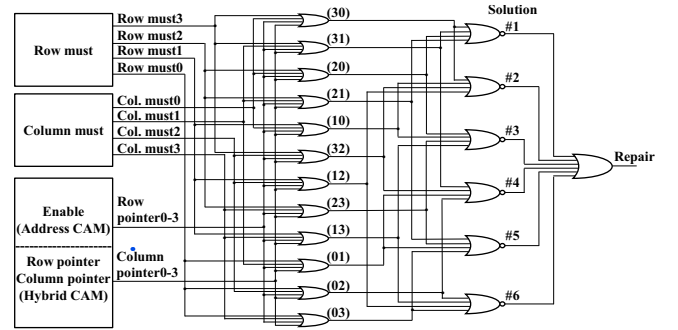


Fig. 10. Block diagram for repair analysis. ( $R_s, C_s = [2, 2]$ ).

Fig. 9 shows examples of the proposed repair analysis for deriving the repair solution in a state with two spare rows and two spare columns within an  $8 \times 8$  memory. The final results of the matrix conversion are based on the fault information from the Address CAM and Hybrid CAM. As shown in the block diagram for the repair analysis in Figs. 10 and 16 signals are designed using combinational logic gates, only the fault-free areas are ultimately checked in the six repair solution lists to determine if all signals in fault-free areas result in a 0 from the NOR gate. In Fig. 9(a), when fault-free areas at (01), (02), (31), and (32) are all verified to be 0, rows 1 and “6,” and columns 4 and 1 are determined as the repair solution through the repair analysis. Similarly, in Fig. 9(b), if fault-free areas at (20), (21), (30), and (31) are all verified to be 0, the repair solution involving rows 6 and 1 and columns 6 and 4 can be derived. Thus, a fast repair solution is derived by exclusively verifying fault-free areas.

### F. CAM Reuse in Online Mode

Fig. 11 shows the flowchart of write and read operations in online mode. The process begins during the power-up sequence, where fault information is loaded from the eFuses, followed by the Address CAM being updated with this information. Once HBM is ready for operation, commands, addresses, and data are input into HBM, and both normal

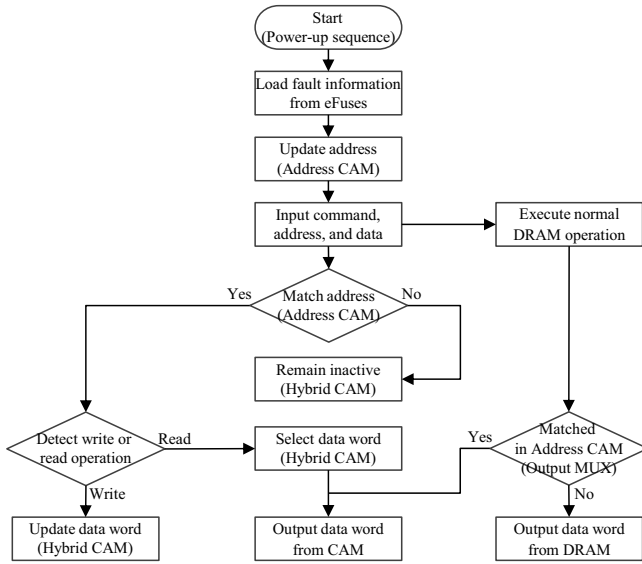


Fig. 11. Flow chart of write and read operations with CAM reuse in online mode.

DRAM operations and CAM reuse functions proceed in parallel. For write operations, if the input address is matched with any stored address in the Address CAM, the input data word is updated in the Hybrid CAM, along with the pointer information in the Address CAM. For read operations, if an address match is detected in the Address CAM, the data word stored in the Hybrid CAM, corresponding to the pointer information in the Address CAM, is selected and transmitted to the output multiplexer, where it is selected in place of the data word from the DRAM. However, if no address match is found, the Hybrid CAM remains inactive, as the input address corresponds to fault-free cells in the DRAM.

The process of matching the input address with fault addresses in the Address CAM and updating additional information in the Hybrid CAM is consistent in both offline and online modes. This consistency enables an efficient design that minimizes area overhead. Finally, utilizing RECAM to handle additional faults enhances HBM's reliability.

### G. Examples of CAM Reuse Operations

For the purpose of CAM reuse, the additional faults (7, 5) and (6, 6) were collected, as shown in Fig. 5. Subsequently, Fig. 12 provides a detailed example of CAM reuse operation in online mode, demonstrating the utilization of these collected addresses. Fig. 12(a) illustrates the process operation sequence, while Fig. 12(b)–(g) describes the operation of the Address CAM and the Hybrid CAM in sequential order. Fig. 12(b) shows that at the start of the power-up sequence, fault information is loaded from the eFuses and transmitted to the Address CAM, which is then updated. Fig. 12(c) illustrates the write operation process where the input address matches one of the fault addresses stored in the Address CAM. The input address (7, 5) is compared with the fault addresses in the Address CAM. If a match is detected for (7, 5), the input data word (10101010) is updated to the Hybrid CAM. Fig. 12(d) illustrates the read operation process when the input address

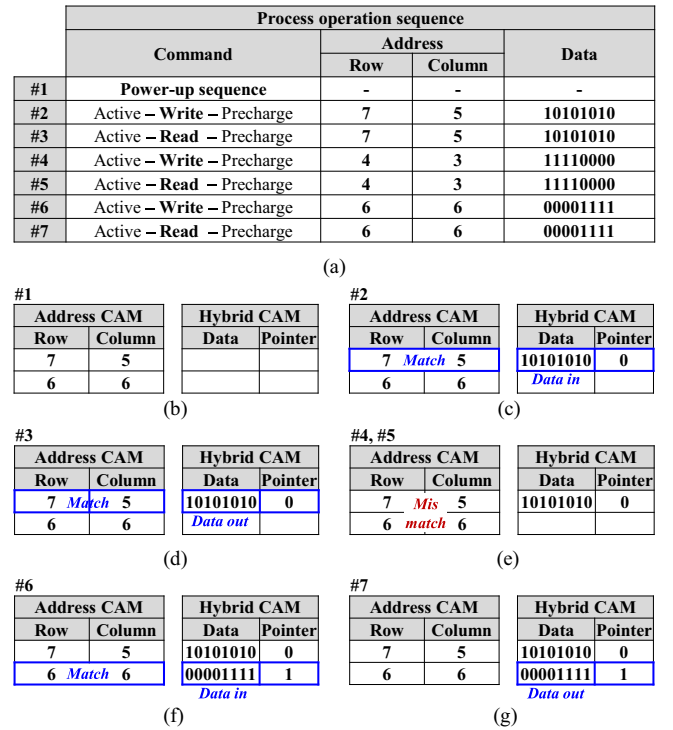


Fig. 12. Examples of CAM reuse operations in online mode. (a) Process operation sequence. (b) Load fault information. (c) Match address during write operation. (d) Match address during read operation. (e) Mismatch address. (f) Match address during write operation. (g) Match address during read operation.

matches one of the fault addresses stored in the Address CAM. The input address (7, 5) is again compared with the stored fault addresses. If a match is found, the data word (10101010) previously stored in the Hybrid CAM is selected and transferred to the output multiplexer, replacing the data word. Fig. 12(e) illustrates the process where the input address does not match any of the fault addresses during both write and read operations. Since the input address (4, 3) is not a fault address, the Hybrid CAM remains inactive, and the DRAM operates normally. Fig. 12(f) presents the same write operation process as Fig. 12(c), where the input address (6, 6) is matched with a fault address in the Address CAM, and the corresponding data word (00001111) is updated to the Hybrid CAM. Fig. 12(g) presents the same read operation process as Fig. 12(d). The input address (6, 6) is compared with the stored fault addresses, and if a match is found, the data word (00001111) stored in the Hybrid CAM is selected and transferred to the output multiplexer, replacing the data word.

Fig. 13 shows an example of write and read operations with CAM reuse in online mode. For example, in offline mode, two additional pivot faults were detected at (5, 2) and (84, 12) through BISR. This fault information was stored by the eFuse programming. During the power-up sequence in online mode, the fault information stored in the eFuses is updated in the Address CAM. When a write command with the address (5, 2) and channel 0 is input by the host, the input address is compared with the stored fault addresses in the Address CAM. If a match is found, the input data (10010001) and pointer

TABLE II  
REPAIR RATE COMPARISON FOR VARIOUS BISRS

| Algorithms | $R_s, C_s = [2, 2]$ |       |      |      |      |      |      |      |      | $R_s, C_s = [3, 3]$ |       |       |       |      |      |      |      |      | $R_s, C_s = [4, 4]$ |       |       |       |       |       |       |       |       |
|------------|---------------------|-------|------|------|------|------|------|------|------|---------------------|-------|-------|-------|------|------|------|------|------|---------------------|-------|-------|-------|-------|-------|-------|-------|-------|
|            | Number of faults    |       |      |      |      |      |      |      |      | Number of faults    |       |       |       |      |      |      |      |      | Number of faults    |       |       |       |       |       |       |       |       |
|            | 4                   | 5     | 6    | 7    | 8    | 9    | 10   | 11   | 12   | 6                   | 7     | 8     | 9     | 10   | 11   | 12   | 13   | 14   | 8                   | 9     | 10    | 11    | 12    | 13    | 14    | 15    | 16    |
| CRESTA     | 100.0               | 98.0  | 87.7 | 63.8 | 39.9 | 21.3 | 10.0 | 4.7  | 2.2  | 100.0               | 99.7  | 97.4  | 91.0  | 77.0 | 58.1 | 39.6 | 24.2 | 13.4 | 100.0               | 100.0 | 99.4  | 97.7  | 93.3  | 83.4  | 68.9  | 54.3  | 37.8  |
| ESP        | 100.0               | 98.0  | 84.8 | 59.3 | 35.4 | 18.8 | 8.9  | 4.2  | 2.0  | 100.0               | 99.7  | 97.0  | 88.6  | 72.3 | 52.8 | 34.6 | 20.6 | 11.4 | 100.0               | 100.0 | 99.3  | 97.1  | 91.7  | 79.8  | 63.9  | 48.6  | 32.6  |
| ISF        | 100.0               | 98.0  | 87.7 | 63.8 | 39.9 | 21.3 | 10.0 | 4.7  | 2.2  | 100.0               | 99.7  | 97.4  | 91.0  | 77.0 | 58.1 | 39.6 | 24.2 | 13.4 | 100.0               | 100.0 | 99.4  | 97.7  | 93.3  | 83.4  | 68.9  | 54.3  | 37.8  |
| SFCC       | 100.0               | 98.0  | 87.7 | 63.8 | 39.1 | 20.6 | 9.8  | 4.6  | 2.2  | 100.0               | 99.7  | 97.4  | 91.0  | 77.0 | 58.1 | 39.6 | 24.2 | 13.4 | 100.0               | 100.0 | 99.4  | 97.7  | 93.3  | 83.4  | 68.9  | 54.3  | 37.8  |
| [21]       | 100.0               | 98.0  | 87.7 | 63.8 | 39.9 | 21.3 | 10.0 | 4.7  | 2.2  | 100.0               | 99.7  | 97.4  | 91.0  | 77.0 | 58.1 | 39.6 | 24.2 | 13.4 | 100.0               | 100.0 | 99.4  | 97.7  | 93.3  | 83.4  | 68.9  | 54.3  | 37.8  |
| [22]       | 100.0               | 98.0  | 87.7 | 63.8 | 39.9 | 21.3 | 10.0 | 4.7  | 2.2  | 100.0               | 99.7  | 97.4  | 91.0  | 77.0 | 58.1 | 39.6 | 24.2 | 13.4 | 100.0               | 100.0 | 99.4  | 97.7  | 93.3  | 83.4  | 68.9  | 54.3  | 37.8  |
| [23]       | 100.0               | 100.0 | 97.3 | 92.7 | 85.1 | 75.2 | 61.1 | 44.5 | 27.9 | 100.0               | 100.0 | 99.6  | 98.0  | 95.4 | 91.9 | 86.7 | 80.5 | 73.4 | 100.0               | 100.0 | 99.9  | 99.5  | 98.7  | 97.2  | 94.8  | 91.9  | 88.0  |
| [24]       | 100.0               | 98.0  | 87.7 | 63.8 | 39.9 | 21.3 | 10.0 | 4.7  | 2.2  | 100.0               | 99.7  | 97.4  | 91.0  | 77.0 | 58.1 | 39.6 | 24.2 | 13.4 | 100.0               | 100.0 | 99.4  | 97.7  | 93.3  | 83.4  | 68.9  | 54.3  | 37.8  |
| RECAM      | 100.0               | 100.0 | 99.9 | 99.7 | 99.5 | 96.7 | 86.3 | 64.9 | 41.1 | 100.0               | 100.0 | 100.0 | 100.0 | 99.9 | 99.9 | 99.9 | 99.5 | 97.4 | 100.0               | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 |

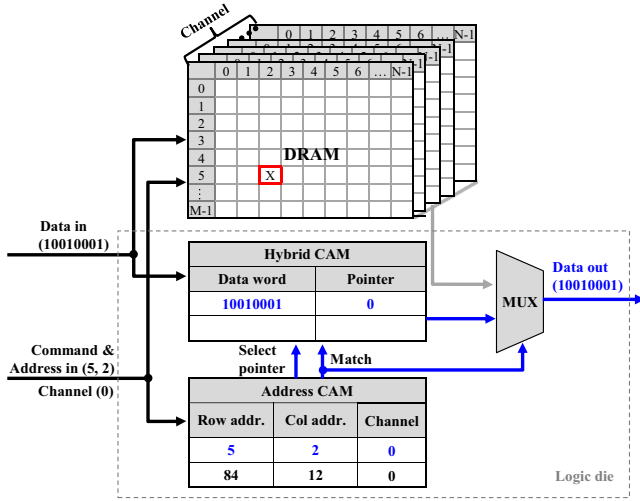


Fig. 13. Example of write and read operations with CAM reuse in online mode.

information are updated in the Hybrid CAM. Subsequently, when a read command with the address (5, 2) and channel 0 is issued, if the address is matched with the one in the Address CAM, the data (10010001) corresponding to the pointer is retrieved from the Hybrid CAM and transmitted to the output multiplexer, where it is selected in place of the faulty data from the DRAM. This ensures that only correct data is processed.

#### IV. EXPERIMENTAL RESULTS

To evaluate the repair rate and analysis time of RECAM, a memory repair simulation tool was used. This tool simulated large memory arrays and performed repair analysis to evaluate the effectiveness of the proposed idea. In the experiment, the number of memories was set to 10 000, with each memory having a size of  $1024 \times 1024$ . Then, the influence on the repair rate and the analysis time was compared by increasing the number of faults while setting the spare rows and spare columns to 2–4, respectively. The fault distribution, modeled after real memory scenarios, was established based on the *ITC'2020 RA Benchmarks* [30]. When faults are individually

distributed, there is a 20% probability of a cluster fault occurring after the previous fault, a 30% probability of a line fault occurring, and the remaining probability represents random faults. The experimental comparisons were conducted against the algorithms of various BISRs. The hardware was synthesized using the 32 nm Synopsys library database [31] and the area overhead is represented in terms of the number of NAND gates.

##### A. Repair Rate

Table II provides the repair rate as a percentage, based on the results of 10 000 iterations where faults are induced in a single channel memory. In the experimental results, the variation in repair rate is evaluated by increasing both the number of spares and the number of faults. When the number of faults is small, the existing spares can repair them, resulting in a 100% repair rate. However, as the number of faults increases and becomes more complex, the redundancy resources become insufficient, leading to a decrease in the repair rate. Additionally, as the number of spares increases, the availability of resources increases, leading to a higher repair rate.

Compared to the method proposed in [23], a higher repair rate is achieved by utilizing RECAM. In [23], additional SRAM redundancy resources are added to the logic die to improve the repair rate, but only pivot faults can be stored to minimize area overhead. However, in RECAM, both pivot and nonpivot faults are collected, and the CAM is reused as redundancy resources, enabling the collection of additional pivot faults. Utilizing RECAM allows for the collection of more faults and achieves a higher repair rate compared to various other BISRs. The results from the repair rate experiments highlight the significance of redundancy resource shortages as the number of faults increases, and it has been demonstrated that reusing CAM can serve as additional redundancy resources.

Fig. 14 shows the comparison of the repair rate based on the increase in channels and the reuse of CAM. The repair rate represents the percentage of repaired memories out of the total tested memories, with the requirement that all channels must be fully composed of repaired memories.



TABLE III  
CLOCK CYCLE COMPARISON FOR VARIOUS BISRS

| Algorithms | $R_s, C_s = [2, 2]$ |      |      |       |       |      |      |      |      | $R_s, C_s = [3, 3]$ |      |      |       |       |      |      |      |      | $R_s, C_s = [4, 4]$ |      |       |       |       |      |      |      |       |
|------------|---------------------|------|------|-------|-------|------|------|------|------|---------------------|------|------|-------|-------|------|------|------|------|---------------------|------|-------|-------|-------|------|------|------|-------|
|            | Number of faults    |      |      |       |       |      |      |      |      | Number of faults    |      |      |       |       |      |      |      |      | Number of faults    |      |       |       |       |      |      |      |       |
|            | 4                   | 5    | 6    | 7     | 8     | 9    | 10   | 11   | 12   | 6                   | 7    | 8    | 9     | 10    | 11   | 12   | 13   | 14   | 8                   | 9    | 10    | 11    | 12    | 13   | 14   | 15   | 16    |
| CRESTA     | 0.0                 | 0.0  | 0.0  | 0.0   | 0.0   | 0.0  | 0.0  | 0.0  | 0.0  | 0.0                 | 0.0  | 0.0  | 0.0   | 0.0   | 0.0  | 0.0  | 0.0  | 0.0  | 0.0                 | 0.0  | 0.0   | 0.0   | 0.0   | 0.0  | 0.0  | 0.0  |       |
| ESP        | 0.0                 | 0.0  | 0.0  | 0.0   | 0.0   | 0.0  | 0.0  | 0.0  | 0.0  | 0.0                 | 0.0  | 0.0  | 0.0   | 0.0   | 0.0  | 0.0  | 0.0  | 0.0  | 0.0                 | 0.0  | 0.0   | 0.0   | 0.0   | 0.0  | 0.0  | 0.0  |       |
| ISF        | 9.0                 | 22.7 | 73.5 | 256.5 | 844.8 | 2937 | 11K  | 40K  | 145K | 14.8                | 31.0 | 87.7 | 268.4 | 834.0 | 2426 | 7014 | 20K  | 66K  | 22.9                | 43.7 | 110.8 | 329.6 | 994.8 | 3055 | 8715 | 22K  | 60K   |
| SFCC       | 3.8                 | 4.4  | 6.2  | 8.3   | 10.5  | 12.7 | 17.6 | 27.9 | 48.7 | 4.7                 | 5.2  | 7.3  | 11.5  | 17.5  | 26.6 | 35.6 | 46.4 | 44.8 | 5.7                 | 6.2  | 8.5   | 13.5  | 22.7  | 38.8 | 64.6 | 99.7 | 146.3 |
| [21]       | 2.3                 | 2.6  | 3.2  | 4.0   | 4.9   | 5.9  | 7.0  | 7.5  | 7.8  | 2.4                 | 2.6  | 3.3  | 4.4   | 6.1   | 8.0  | 10.3 | 12.2 | 14.0 | 2.5                 | 2.7  | 3.4   | 4.9   | 7.8   | 11.3 | 16.2 | 21.4 | 26.4  |
| [22]       | 1.4                 | 1.5  | 1.7  | 2.1   | 2.9   | 4.4  | 5.8  | 6.7  | 7.1  | 2.5                 | 2.9  | 3.6  | 4.6   | 5.6   | 6.5  | 7.6  | 9.1  | 11.3 | 5.1                 | 6.2  | 8.5   | 11.3  | 14.6  | 17.5 | 20.9 | 24.7 | 27.0  |
| [23]       | 2.1                 | 2.3  | 2.5  | 2.9   | 3.3   | 3.5  | 3.8  | 3.9  | 3.9  | 2.4                 | 2.7  | 2.9  | 3.1   | 3.4   | 3.8  | 4.0  | 4.3  | 4.5  | 2.9                 | 3.1  | 3.3   | 3.5   | 3.7   | 4.0  | 4.3  | 4.6  | 4.9   |
| [24]       | 0.0                 | 0.0  | 0.0  | 0.0   | 0.0   | 0.0  | 0.0  | 0.0  | 0.0  | 0.0                 | 0.0  | 0.0  | 0.0   | 0.0   | 0.0  | 0.0  | 0.0  | 0.0  | 0.0                 | 0.0  | 0.0   | 0.0   | 0.0   | 0.0  | 0.0  | 0.0  |       |
| RECAM      | 0.0                 | 0.0  | 0.0  | 0.0   | 0.0   | 0.0  | 0.0  | 0.0  | 0.0  | 0.0                 | 0.0  | 0.0  | 0.0   | 0.0   | 0.0  | 0.0  | 0.0  | 0.0  | 0.0                 | 0.0  | 0.0   | 0.0   | 0.0   | 0.0  | 0.0  | 0.0  |       |

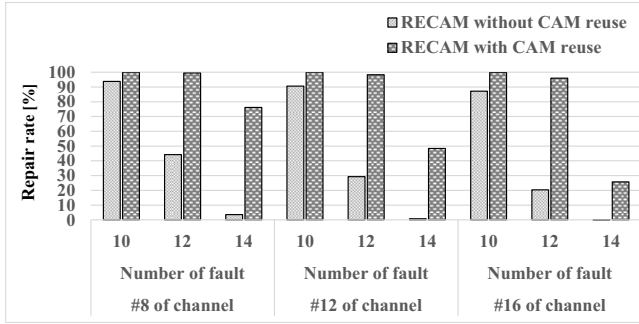


Fig. 14. Comparison of repair rate with increasing channels. ( $R_s, C_s = [4, 4]$ ).

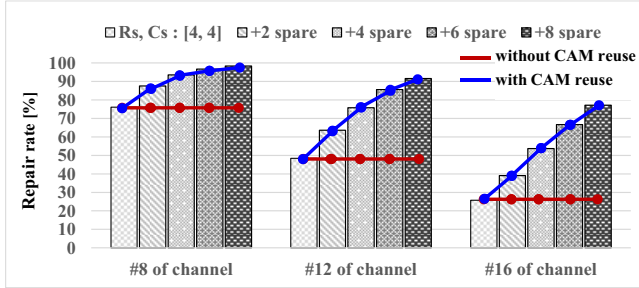


Fig. 15. Comparison of repair rate with increasing CAM size for 14 faults. ( $R_s, C_s = [4, 4]$ ).

This is based on the occurrence of the same number of faults in each channel, sharing the redundancy resources of CAM across the entire set of channels. Therefore, as the number of channels increases, the repair rate decreases. The reuse of CAM provides more resources for redundancy, which becomes increasingly beneficial as the number of channels and faults increases. Consequently, this approach achieves a repair rate that is up to several hundred times higher compared to without the reuse of CAM.

The experimental results in Fig. 15 show the comparison of repair rates with increasing CAM size, while fixing  $R_s = 4$  and  $C_s = 4$ . The repair rate differs significantly based on whether the CAM is reused as redundancy resources, with increasing CAM sizes leading to an even greater impact by

enabling the storage and repair of more faults. Particularly, redundancy resources in the CAM are available for all channels, enabling efficient repair when faults occur in a specific channel. Consequently, increasing the CAM size by utilizing the available area in the logic die can enhance the repair rate. This approach is more efficient than adding spare lines to all channels in the DRAM, as doing so increases the DRAM size and raises production costs.

### B. Analysis Time

Table III presents the results of comparing the average clock cycle values for various BISRs. The analysis time is measured in terms of the clock cycles required to derive the repair solution after the test sequences are completed. Additional clock cycles are required by the hybrid BIRA as the repair analysis is performed after the fault collection process. ISF, SFCC, [21], [22], and [23] show that with a small number of faults, the fault types are simple, enabling the repair solution to be derived in fewer clock cycles. Conversely, as the number of faults increases, the fault types become more complex, leading to an increase in clock cycles. The repair analysis and fault collection are conducted concurrently in dynamic BIRA, including CRESTA, ESP, [24], and RECAM, enabling the immediate derivation of the repair solution. As a result of eliminating the need for additional analysis time, the total test time decreases, leading to a substantial reduction in test costs.

### C. Area Overhead

In the experiment, the memory had a size of  $8192 \times 8192$  with two spares each for rows and columns. The hardware was synthesized to compare the area overhead in relation to the increase in memory size against the representative hybrid BIRA [21] and dynamic BIRA [24], which are known for their excellent overall performance in BISR. Additionally, comparisons were conducted to measure the increment associated with the reuse of CAM, and the area overhead was evaluated for [25] and [26], facilitating a broader comparison of various BISR approaches. Fig. 16 shows the reuse of CAM results in only a 3% increase in the gate count, which helps to reduce the burden of area overhead by utilizing existing circuits.

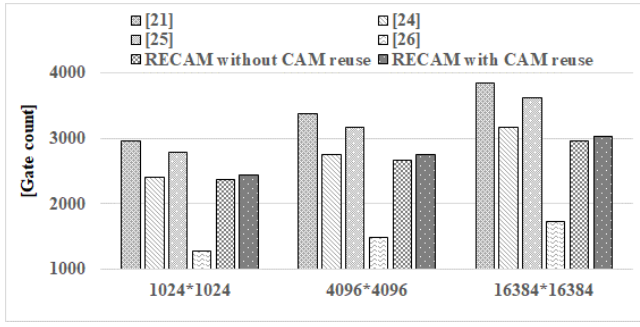


Fig. 16. Comparison of area overhead with increasing memory sizes. ( $R_s$ ,  $C_s = [2, 2]$ ).

TABLE IV  
ERROR CORRECTION CAPABILITIES

| Reed-Solomon type | Symbol size (bits) | Fault mode |            |            |
|-------------------|--------------------|------------|------------|------------|
|                   |                    | Single bit | Double bit | Triple bit |
| [5]               | 16                 | 100%       | 4.95%      | 0.23%      |
| [6]               | 8                  | 100%       | 52.48%     | 3.54%      |

In comparisons based on the increase in memory size, an increase of over 30% is observed for [21], [24], [25], and [26] when scaling from  $1024 \times 1024$  to  $16384 \times 16384$ , whereas a smaller increase of approximately 24% is observed for RECAM. This smaller increase in area overhead for RECAM is attributed to its use of pointer information for repair analysis, in contrast to [21], [24], [25], and [26], which rely on address-based analysis. Meanwhile, [26] shows significantly lower overhead due to the storage of fault information in a fault-free region of DRAM. However, identifying and controlling this fault-free region is highly complex and challenging. When comprehensively considering the increase in area overhead, utilizing RECAM demonstrates a significant advantage in minimizing area overhead as memory size increases.

#### D. Online Performance

In online mode, the process of matching addresses and transferring the data word can be completed within 2–3 clock cycles. Additionally, the output multiplexer for the data out is added when the parallel operation of DRAM and RECAM is performed, as shown in Fig. 13. To evaluate the delay of the output multiplexer, the synthesis was conducted using the 32-nm Synopsys library. Although the delay due to the output multiplexer in the critical path is 30 ps, this delay is negligible since the operation of RECAM is significantly faster than the read latency of HBM3, which has a minimum specification of 4 n clock cycles at low frequency and a maximum of 63 n clock cycles at high frequency, providing sufficient time for data processing within the logic die.

In HBM, a two-tier ECC structure is adopted to enhance the reliability, consisting of on-die ECC [5], [6] and the system ECC [32], [33]. ECCs can correct soft errors, which are temporary and include issues like data interference, within its correction capacity, as shown in Table IV. The proposed idea aims to repair hard faults by reusing the CAM as redundancy resources, utilizing the detection function of the system ECC

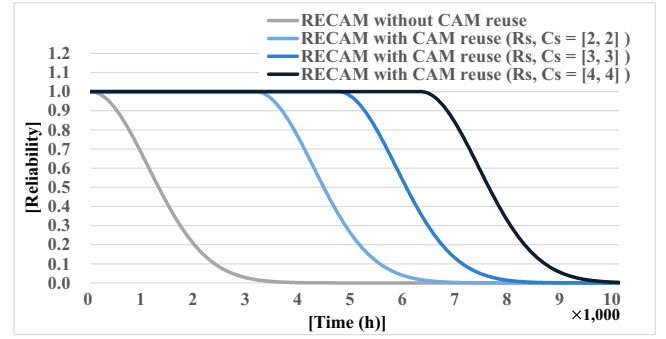


Fig. 17. Reliability of  $128K \times 256$  DRAM with CAM reuse.

in online mode. For instance, when the system ECC detects and corrects an error, it stores the address in a buffer to track the frequency of the error occurring. If the error count at a specific address exceeds a set threshold, it is considered a hard fault, and the corresponding fault information is updated in the Address CAM to be utilized for redundancy. Since both the CAM and the system ECC are located within the logic die, hard fault information can be transferred from the system ECC to the Address CAM in real time. Consequently, reusing the CAM to repair hard faults enhances memory reliability.

The Poisson distribution, extensively applied in reliability evaluations, was utilized to derive reliability functions [34], [35], [36]. Models simulating the accumulation of hard errors over time were used, and previously established methods for reliability evaluations were applied to assess the reliability resulting from CAM reuse. Fig. 17 illustrates the reliability of  $128K \times 256$  DRAM by utilizing RECAM. As time progresses, the reliability without CAM reuse degrades significantly. However, the reliability can substantially increase with redundancy resources provided by CAM reuse.

#### E. Overall Evaluation

Fig. 18 shows the comparison results of overall performance in repair rate, clock cycle, and area overhead. As the number of faults increases, more faults can be repaired by utilizing RECAM without requiring additional analysis time to derive the repair solution, thereby providing a significant overall advantage in manufacturing and test costs for HBM. In addition, the burden on the area overhead is significantly alleviated by the reuse of CAM. Consequently, utilizing RECAM is observed to be closest to the ideal BISR. Therefore, outstanding performance is demonstrated by utilizing RECAM under conditions with increasing numbers of faults in memory, compared to CAM-based BISRs.

In online mode, the delay caused by adding the output multiplexer is negligible. This is because the operation of RECAM is significantly faster than the read latency of HBM3. Additionally, the reliability of HBM can be substantially improved by utilizing RECAM.

#### V. CONCLUSION

BISR is crucial for developing high-capacity and reliable memories. This study demonstrates that reusing CAM significantly improves fault collection and repair rate efficiency

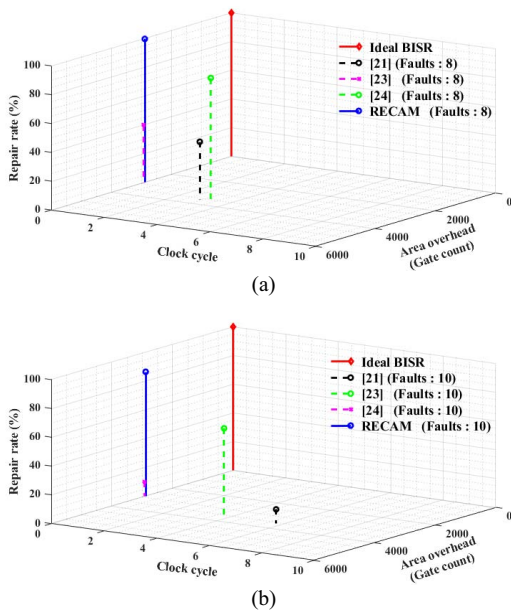


Fig. 18. Comparison of overall performance. (a) Number of faults: 8. (b) Number of faults: 10. (Memory size:  $8192 \times 8192$ ,  $R_s$ ,  $C_s = [2, 2]$ ).

compared to CAM-based BISRs. Zero clock cycle overhead is maintained by the dynamic BIRA, and the area overhead increases by only 3% due to the reuse of CAM. Experimental results indicate that utilizing RECAM comes closest to achieving the ideal BISR, as it demonstrates outstanding performance in environments with increasing numbers of faults. Furthermore, the reuse of CAM enhances the reliability of HBM by providing additional redundancy resources.

## REFERENCES

- [1] H. Jun et al., "HBM (high bandwidth memory) DRAM technology and architecture," in *Proc. IEEE Int. Memory Workshop (IMW)*, Monterey, CA, USA, 2017, pp. 1–4.
- [2] J. C. Lee et al., "High Bandwidth Memory(HBM) with TSV technique," in *Proc. Int. SoC Des. Conf.*, 2016, pp. 181–182.
- [3] D. U. Lee et al., "25.2 A 1.2V 8Gb 8-channel 128GB/s High-Bandwidth Memory (HBM) stacked DRAM with effective microbump I/O test methods using 29nm process and TSV," in *Proc. IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers*, 2014, pp. 432–433.
- [4] D. U. Lee et al., "A 1.2 V 8 GB 8-channel 128 GB/s High-Bandwidth Memory (HBM) stacked DRAM with effective I/O test circuits," *IEEE J. Solid-State Circuits*, vol. 50, no. 1, pp. 191–203, Jan. 2015.
- [5] M.-J. Park et al., "A 192-Gb 12-high 896-GB/s HBM3 DRAM with a TSV Auto-calibration scheme and machine-learning-based layout optimization," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 256–269, Jan. 2023.
- [6] Y. Ryu et al., "A 16 GB 1024 GB/s HBM3 DRAM with source-synchronized bus design and on-die error control scheme for enhanced RAS features," *IEEE J. Solid-State Circuits*, vol. 58, no. 4, pp. 1051–1061, Apr. 2023.
- [7] J. Lee et al., "13.4 A 48GB 16-High 1280GB/s HBM3E DRAM with all-around power TSV and a 6-phase RDQS scheme for TSV area optimization," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, San Francisco, CA, USA, 2024, pp. 238–240.
- [8] D. U. Lee et al., "Design considerations of HBM stacked DRAM and the memory architecture extension," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, San Jose, CA, USA, 2015, pp. 1–8.
- [9] H. Jun, S. Nam, H. Jin, J.-C. Lee, Y. J. Park, and J. J. Lee, "High-Bandwidth Memory (HBM) test challenges and solutions," *IEEE Des. Test.*, vol. 34, no. 1, pp. 16–25, Feb. 2017.
- [10] K. Cho, W. Kang, H. Cho, C. Lee, and S. Kang, "A survey of repair analysis algorithms for memories," *ACM Comput. Surveys*, vol. 49, no. 3, pp. 1–41, Dec. 2016.
- [11] S.-Y. Kuo and W. K. Fuchs, "Efficient spare allocation in reconfigurable arrays," in *Proc. 23rd ACM/IEEE Design Autom. Conf.*, Las Vegas, NV, USA, Jun./Jul. 1986, pp. 385–390.
- [12] H.-Y. Lin, F.-M. Yeh, and S.-Y. Kuo, "An efficient algorithm for spare allocation problems," *IEEE Trans. Rel.*, vol. 55, no. 2, pp. 369–378, Jun. 2006.
- [13] H. Cho, W. Kang, and S. Kang, "A fast redundancy analysis algorithm in ATE for repairing faulty memories," *ETRI J.*, vol. 34, no. 3, pp. 478–481, Jun. 2012.
- [14] H. Cho, W. Kang, and S. Kang, "A very efficient redundancy analysis method using fault grouping," *ETRI J.*, vol. 13, no. 3, pp. 439–447, Jun. 2013.
- [15] H. Lee, K. Cho, D. Kim, and S. Kang, "Fault group pattern matching with efficient early termination for high-speed redundancy analysis," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 7, pp. 1473–1482, Jul. 2018.
- [16] T. H. Kim, H. Lee, and S. Kang, "GPU-based redundancy analysis using concurrent evaluation," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 3, pp. 805–817, Mar. 2020.
- [17] T. Kawagoe, J. Ohtani, M. Niino, T. Oishi, M. Hamada, and H. Hidaka, "A built-in self-repair analyzer (CRESTA) for embedded DRAMs," in *Proc. Int. Test Conf.*, Oct. 2000, pp. 567–574.
- [18] C.-T. Huang, C.-F. Wu, J.-F. Li, and C.-W. Wu, "Built-in redundancy analysis for memory yield improvement," *IEEE Trans. Rel.*, vol. 52, no. 4, pp. 386–399, Dec. 2003.
- [19] P. Ohler, S. Hellebrand, and H.-J. Wunderlich, "An integrated built-in test and repair approach for memories with 2-D redundancy," in *Proc. 12th IEEE Eur. Test Symp.*, Freiburg, Germany, May 2007, pp. 91–96.
- [20] W. Jeong, I. Kang, K. Jin, and S. Kang, "A fast built-in redundancy analysis for memories with optimal repair rate using a line-based search tree," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 17, no. 12, pp. 1665–1678, Dec. 2009.
- [21] W. Jeong, J. Lee, T. Han, K. Lee, and S. Kang, "An advanced BIRA for memories with an optimal repair rate and fast analysis speed by using a branch analyzer," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 29, no. 12, pp. 2014–2026, Dec. 2010.
- [22] K. Cho, Y. Lee, S. Seo, and S. Kang, "An efficient BIRA utilizing characteristics of spare pivot faults," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 3, pp. 551–561, Mar. 2019.
- [23] D. Han, H. Lee, and S. Kang, "Effective spare line allocation built-in redundancy analysis with base common spare for yield improvement of 3-D memory," *IEEE Access*, vol. 9, pp. 76716–76729, 2021.
- [24] H. Lee, D. Han, S. Lee, and S. Kang, "Dynamic built-in redundancy analysis for memory repair," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 10, pp. 2365–2374, Oct. 2019.
- [25] H.-H. Liu et al., "A built-off self-repair scheme for channel-based 3-D memories," *IEEE Trans. Comput.*, vol. 66, no. 8, pp. 1293–1301, Aug. 2017.
- [26] C.-H. Oh, S.-E. Kim, and J.-S. Yang, "BIRA with optimal repair rate using fault-free memory region for area reduction," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 64, no. 12, pp. 3160–3171, Dec. 2017.
- [27] K. Pagiamtzis and A. Shekholeslami, "Content-addressable memory (CAM) circuits and architectures: A tutorial and survey," *IEEE J. Solid-State Circuits*, vol. 41, no. 3, pp. 712–727, Mar. 2006.
- [28] A. Benso, S. Chiusano, G. D. Natale, and P. Prinetto, "An on-line BIST RAM architecture with self-repair capabilities," *IEEE Trans. Reliability*, vol. 51, no. 1, pp. 123–128, Mar. 2002.
- [29] A. Scionti, S. Mazumdar, S. D. Carlo, and S. Hamdioui, "SIERRA—Simulation environment for memory redundancy algorithms," *Simul. Model. Pract. Theory*, vol. 69, pp. 14–30, Dec. 2016.
- [30] H. Lee, K. Cho, S. Kang, W. Kang, S. Lee, and W. Jeong, "Fail memory configuration set for RA estimation," in *Proc. Int. Test Conf.*, Nov. 2020, pp. 1–9.
- [31] Synopsys Armenian Education Department (SAED) 32nm Generic Library, Synopsys Inc., Sunnyvale, CA, USA, 2012.
- [32] S. Shukla and N. Bergmann, "Single bit error correction implementation in CRC-16 on FPGA," in *Proc. IEEE Int. Conf. Field-Program. Technol.*, Brisbane, QLD, Australia, 2004, pp. 319–322.
- [33] C. Shi-yi and L. Yu-bai, "Error correcting cyclic redundancy checks based on confidence declaration," in *Proc. 6th Int. Conf. ITS Telecommun.*, Chengdu, China, 2006, pp. 511–514.
- [34] C.-L. Su, Y.-T. Yeh, and C.-W. Wu, "An integrated ECC and redundancy repair scheme for memory reliability enhancement," in *Proc. IEEE Int. Symp. Defect Fault Tolerance VLSI Syst.*, Oct. 2005, pp. 81–89.

- [35] T.-H. Wu et al., "A memory yield improvement scheme combining built-in self-repair and error correction codes," in *Proc. IEEE Int. Test Conf.*, Nov. 2012, pp. 1–9.
- [36] H. Lee, Y. Yoo, S. H. Shin, and S. Kang, "ECMO: ECC architecture reusing content-addressable memories for obtaining high reliability in DRAM," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 6, pp. 781–793, Jun. 2022.



**Youngki Moon** received the B.S. and M.S. degrees in electrical and electronic engineering from Yonsei University, Seoul, South Korea, in 2021 and 2023, respectively, where he is currently pursuing the Ph.D. degree with the Department of Electrical and Electronics Engineering.

His current research interests include error correction code, secure scan design, and design for testability.



**Joonsik Yoon** received the B.S. degree in electronics, communication, and computer engineering from Hanyang University, Seoul, South Korea, in 2011. He is currently pursuing the M.S. degree with the Department of Electrical and Electronics Engineering, Yonsei University, Seoul.

Since 2011, he has been a Product Engineer with SK Hynix Inc., Icheon, South Korea. His current research interests include memory testing, redundancy analysis algorithms, built-in self-testing, and built-in self-repair.



**Seung Ho Shin** received the B.S. degree in electronic engineering from Sogang University, Seoul, South Korea, in 2021. He is currently pursuing the combined Ph.D. degree with the Department of Electrical and Electronic Engineering, Yonsei University, Seoul.

His current research interests include built-in self-testing, built-in self-repair, redundancy analysis algorithms, error detection/correction, reliability, design for testability, and VLSI design.



**Hayoung Lee** received the B.S. and combined Ph.D. degrees in electrical and electronic engineering from Yonsei University, Seoul, South Korea, in 2016 and 2023, respectively.

He is currently an Assistant Professor with the Department of Intelligence Semiconductor Engineering, Ajou University, Suwon, South Korea. His current research interests include design for testability/debug, VLSI/SoC design and testing, reliability, memory test/repair, built-in self-test/repair, and system-level test/validation.



**Sungho Kang** (Senior Member, IEEE) received the B.S. degree in control and instrumentation engineering from Seoul National University, Seoul, South Korea, in 1986, and the M.S. and Ph.D. degrees in electrical and computer engineering from the University of Texas at Austin, Austin, TX, USA, in 1988 and 1992, respectively.

He was a Research Scientist with the Schlumberger Laboratory for Computer Science, Schlumberger Inc., Austin, and a Senior Staff Engineer with Semiconductor Systems Design Technology, Motorola Inc., Austin. Since 1994, he has been a Professor with the Department of Electrical and Electronic Engineering, Yonsei University, Seoul. His current research interests include VLSI/SoC design and testing, design for testability, design for manufacturability, and fault tolerant computing.